

■■ SentinelAI – Technical & Architectural Documentation

Welcome to the official technical documentation for **SentinelAI** (National Digital Public Safety Intelligence Platform). SentinelAI is developed as a production-grade, local-first monolithic platform designed for the **Smart India Hackathon 2024** under the problem statement **AI for Digital Public Safety**.

This document describes the codebase architecture, database models, integrated AI models, and user role mechanics.

■■ System Architecture

SentinelAI utilizes an integrated, multi-role monolith structure designed to simulate instant coordination between citizens, local police, financial institutions, telecom carriers, and administrators.

```
graph TD
    subgraph frontend ["Frontend Application (Next.js 16)"]
        Citizen["Citizen Portal"]
        Police["Police Command Center"]
        Bank["Bank Officer Dashboard"]
        Telecom["Telecom Operator Console"]
        Admin["System Admin Panel"]
    end

    subgraph backend ["Backend Services (FastAPI)"]
        Router["Monolithic API Routers"]
        Auth["JWT Session & Credentials validator"]
        DBConn["SQLAlchemy Engine Context Manager"]
    end

    subgraph ai ["Intelligent AI Pipelines (AI Core)"]
        Voice["KAVACH-VOICE: Scam Text/Voice Classifier"]
        Curr["KAVACH-CURRENCY: Counterfeit Bill Analyzer"]
        Net["KAVACH-NET: GNN-powered Fraud Rings mapper"]
        Geo["KAVACH-GEO: Geospatial Crime forecaster"]
        Predict["KAVACH-PREDICT: XGBoost Mule Velocity scorer"]
    end

    subgraph db_layer ["Database Layer"]
        sqlite["SQLite Engine: kavach.db"]
        neo4j["Neo4j Graph Database"]
        mock["Fallback Neo4j Engine"]
    end

    %% Routing calls
    Citizen -->|HTTPS API Requests| Router
    Police -->|HTTPS API Requests| Router
    Bank -->|HTTPS API Requests| Router
    Telecom -->|HTTPS API Requests| Router
    Admin -->|HTTPS API Requests| Router
```

```

%% Backend Context
Router --> Auth
Auth --> DBConn

%% AI Model Executions
DBConn --> Voice
DBConn --> Curr
DBConn --> Net
DBConn --> Geo
DBConn --> Predict

Voice --> sqlite
Curr --> sqlite
Net --> sqlite
Geo --> sqlite
Predict --> sqlite

Net -->|Bolt Driver / 7687| neo4j
neo4j -->|If Connection Offline| mock

```

■ Repository Directory Layout

The workspace repository organizes the monolith implementation cleanly between services:

```

ET AI/
■■■ .env.example           # Environment variables template for database ports
■■■ .gitignore             # Git excludes for caches, Next.js build files, and venv modules
■■■ package.json           # Root npm scripts to control both developer servers simultaneously
■■■ requirements.txt        # PyPI python modules list for FastAPI and AI models
■■■ database/
■   ■■■ schema.sql         # Core SQL Schemas (optimized for SQLite & PostgreSQL compatibility)
■   ■■■ seed_data.py        # Automatic seeding script for demo database records
■■■ docs/
■   ■■■ ARCHITECTURE.md     # [THIS DOCUMENT] Detailed system architectural guide
■■■ backend/
■   ■■■ main.py             # Main FastAPI server entry point (configures CORS & mounts routes)
■   ■■■ config.py           # Settings controller reading environment configurations
■   ■■■ database/
■       ■■■ connection.py   # SQLAlchemy Connection manager and Session factory
■   ■■■ ai/
■       ■■■ counterfeit_detector.py # YOLOv11 & EfficientNet-B4 counterfeit checker
■       ■■■ fraud_graph.py    # Neo4j and Fallback Mock fraud rings visualizer
■       ■■■ geospatial.py    # H3 Hexagonal indexing & ST-GCN crime map predictions
■       ■■■ predictive.py     # XGBoost Transaction risk and emerging campaign analysis
■       ■■■ scam_detector.py  # IndicBERT + RoBERTa voice/text scam classification
■   ■■■ routers/           # API router files
■       ■■■ auth.py, citizen.py, police.py, bank.py, telecom.py, admin.py, search.py
■   ■■■ schemas/           # Input/Output validation models (Pydantic v2)
■■■ frontend/
■■■ package.json           # Next.js scripts & packages
■■■ postcss.config.mjs     # Tailwind PostCSS configuration
■■■ tsconfig.json          # TypeScript configuration
■■■ app/
■   ■■■ layout.tsx          # Base metadata, navbar wrapper, and styling imports
■   ■■■ page.tsx            # Dynamic Multi-Role Landing page (with optimized 2-row grids)
■   ■■■ analytics/         # Platform-wide security overview statistics
■   ■■■ dashboard/         # Role-based panels (citizen, police, bank, telecom, admin)

```

■ ■ Core Technology Stack

- **Frontend:** Next.js 16 (App Router, TSX) + Tailwind CSS v4 + Framer Motion (premium layout transitions) + Lucide Icons.
- **Backend:** FastAPI (Python 3.12+) + SQLAlchemy (ORM) + Pydantic v2 (Input validation).
- **Database:** SQLite (local single file [kavach.db](#) for serverless development) / PostgreSQL (for production).
- **Relationship Graph:** Neo4j (via bolt protocol [7687](#)) with a custom fallback mock engine.

■ KAVACH Intelligent AI Modules

The backend AI logic simulates state-of-the-art architectures optimized for local host speed:

AI Module	Code File	Simulates Model Stack	Objective
KAVACH-VOICE	scam_detector.py	IndicBERT, Whisper v3, ECAPA-TDNN	Detects digital arrest, banking, utility, and courier scams in 12 Indian languages.
KAVACH-CURRENCY	counterfeit_detector.py	YOLOv11, EfficientNet-B4, ViT-B/16	Scans security thread, watermark, UV marks, and serial numbers.
KAVACH-NET	fraud_graph.py	GraphSAGE, GAT, Neo4j Bolt	Traces multi-hop transaction networks to find fraud ring endpoints.
KAVACH-GEO	geospatial.py	ST-GCN, Prophet, H3 Hexagons	Predicts crime hotspots and automatically suggests optimal patrol routes.
KAVACH-PREDICT	predictive.py	XGBoost Ensemble, Isolation Forest	Detects mule bank accounts, transaction velocities, and alerts target victims.

■ ■ Database Tables Schema

The SQLite schema in [kavach.db](#) structures information via the following main entities:

1. [users](#): Main system actors. Hashed with SHA-256 for secure mobile lookup, containing user role access variables.
2. [police_officers](#): Connected metadata for law enforcement users including badges, ranks, and police station locations.

3. `fraud_cases`: FIR-style reported scams containing case status, risk levels, and automated case summaries.
4. `evidence`: Digital evidence records uploaded by citizens (audio transcriptions, SMS texts, bill screenshots).
5. `fraud_actors`: Tracked phone numbers, bank accounts, and IMEI codes suspected of hosting scam campaigns.
6. `geospatial_crime_events`: Categorized coordinates tracked under H3 index cells for crime mapping.
7. `audit_logs`: System logs tracking administrative and investigative actions.

■ API Reference (Endpoints)

All endpoints reside under `/api/v1` prefix:

- **Authentication (`/auth`):**
 - `POST /register`: Registers new users with specialized roles.
 - `POST /login`: Validates password and generates JWT access tokens.
 - `POST /verify-otp`: Demo verification step using default code `123456`.
- **Citizen (`/citizen`):**
 - `GET /alerts`: Returns local security advisories.
 - `POST /report`: Uploads fraud evidence (text, audio, image).
- **Police (`/police`):**
 - `GET /dashboard`: Overview statistics on active crime investigations.
 - `GET /cases`: Retrieves full list of active/resolved cases.
 - `GET /network-graph`: Feeds the interactive fraud ring visualizer.
- **Bank (`/bank`):**
 - `GET /transactions/high-risk`: Yields suspicious accounts flagged for hold.
 - `POST /mule/freeze`: Freezes money mule accounts immediately.
- **Telecom (`/telecom`):**
 - `GET /numbers/flagged`: Identifies numbers broadcasting spam voice/SMS.
 - `POST /numbers/block`: Simulates SIM disconnection and IMEI blacklisting.